

WEEK – 1, 2

Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

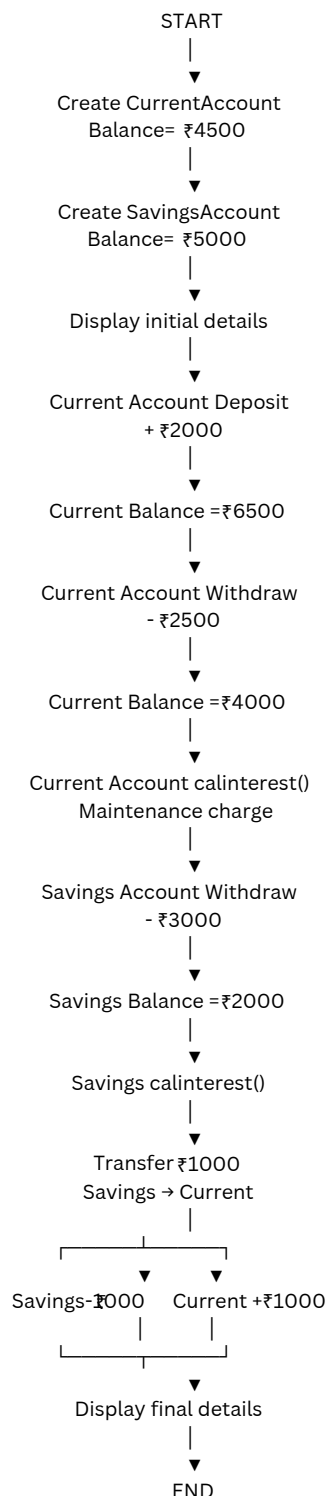
Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfer

Steps of program :



Step 1: Start.

Step 2: Define the Account class with:

- account number
- account holder
- balance

Step 3: Create a constructor in Account to initialize account number, account holder and balance.

Step 4: Define getter methods to access private account details.

Step 5: Define deposit() to add money to the account.

Step 6: Define withdraw() and calinterest() methods which can be overridden by child classes.

Step 7: Define transfer() to transfer money from one Account object to another Account object.

Step 8: Define SavingsAccount as a subclass of Account.

Step 9: Override withdraw() in SavingsAccount to check the balance and withdraw money.

Step 10: Override calinterest() in SavingsAccount to calculate and add interest.

Step 11: Define CurrentAccount as a subclass of Account.

Step 12: Override withdraw() in CurrentAccount to allow withdrawal up to the overdraft limit.

Step 13: Override calinterest() in CurrentAccount to apply the maintenance charge.

Step 14: In main(), create a CurrentAccount object with account number, holder name and initial balance.

Step 15: Create a SavingsAccount object with account number, holder name and initial balance.

Step 16: Display the initial account details.

Step 17: Deposit ₹2000 into the current account.

Step 18: Withdraw ₹2500 from the current account.

Step 19: Calculate the current account charge.

Step 20: Withdraw ₹3000 from the savings account.

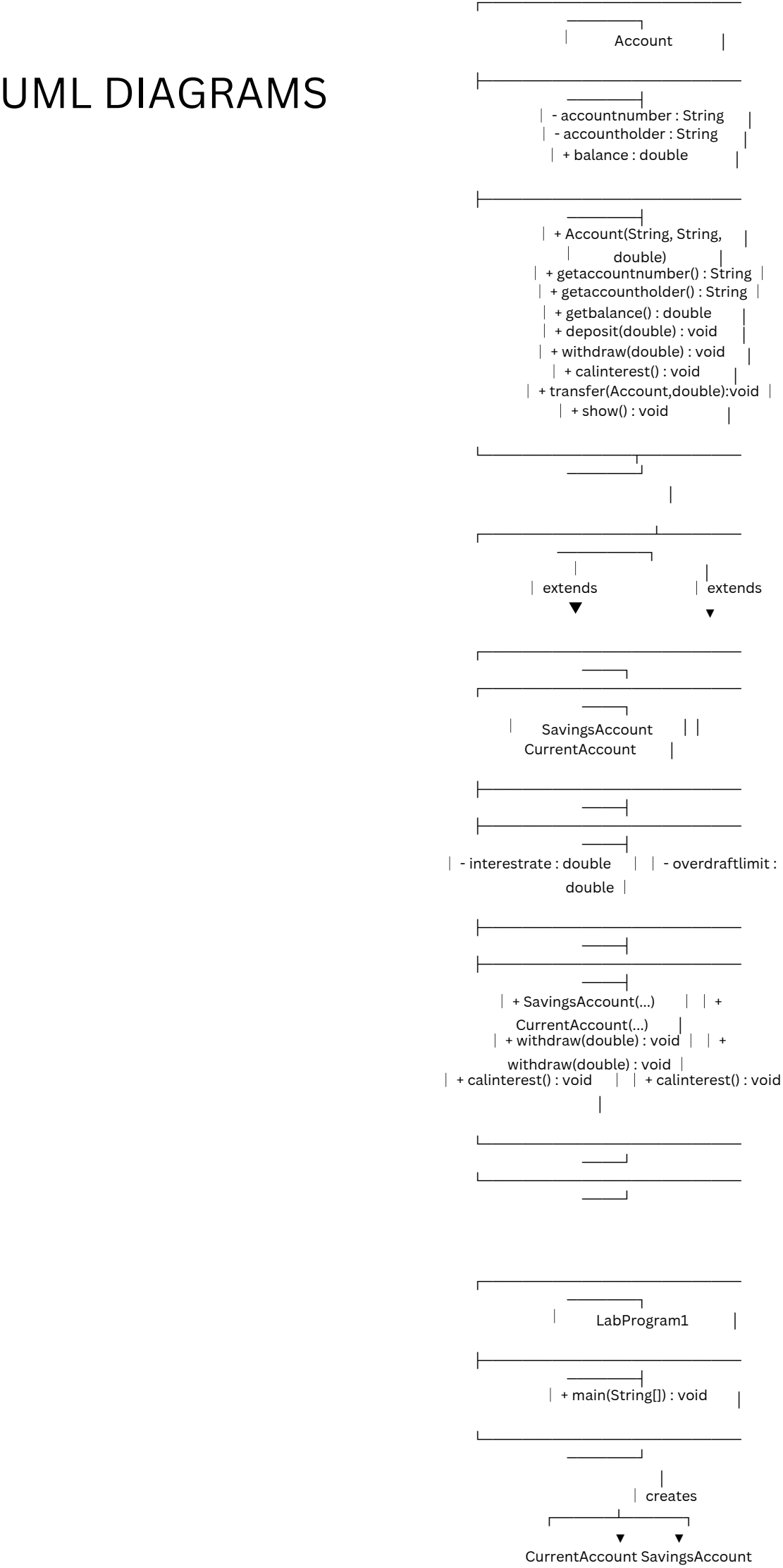
Step 21: Calculate the savings account interest.

Step 22: Transfer ₹1000 from the savings account to the current account.

Step 23: Display the final account details.

Step 24: Stop.

UML DIAGRAMS



```

        class Account{
private String accountnumber; // details are hide
        private String accountholder;
        public double balance;
        public Account(String accountnumber , String
        accountholder ,double balance){
        this.accountnumber = accountnumber; //using
        constructors for parameters passing
        this.accountholder = accountholder;
        this.balance = balance;
        }
        public String getaccountnumber() // doing
        encapsulation by setters and getters
        {
        return accountnumber;
        }
        public String getaccountholder(){
        return accountholder;
        }
        public double getbalance(){
        return balance;
        }
        void deposit (double amount){
        if(amount <= 0){
System.out.println("deposit amount is greater than
        zero");
        return;
        }
        balance += amount;
System.out.println(" the deposited amount is " +
        amount);
        }
        void withdraw (double amount){
        };
        void calinterest(){};
        void transfer(Account fixedAccount , double amount){
        System.out.println("transfer
of"+amount+"from"+accountnumber+"to"+fixedAccount
        .getaccountnumber()+"---");
        this.withdraw(amount);
        fixedAccount.deposit(amount);
        }
        void show(){
System.out.println("ACCOUNT NUMBER IS " +
        accountnumber);
System.out.println("ACCOUNT HOLDER IS " +
        accountholder);
System.out.println("BALANCE IS " + balance);
        }
        }
        class SavingsAccount extends Account {
        private double interestrate;
        public SavingsAccount(String accountnumber , String
        accountholder , double balance){
        super(accountnumber, accountholder, balance);
        this.interestrate = interestrate;
        }
        @Override
        void withdraw(double amount){
        if(amount <= 0){
System.out.println("with draw amount is not
        possible ");
        }else if (amount >= balance){
System.out.println("it has insufficient balance");
        }else{
        balance -= amount;
System.out.println("withdraw is " + amount );
        }
        }
        @Override

```

```

    };
    void calinterest();
    void transfer(Account fixedAccount , double amount){
System.out.println("transfer of"+amount+"from"+accountnumber+"to"+fixedAccount.getaccountnumber()+"-
--");
        this.withdraw(amount);
        fixedAccount.deposit(amount);
    }
    void show(){
System.out.println("ACCOUNT NUMBER IS " + accountnumber);
System.out.println("ACCOUNT HOLDER IS " + accountholder);
System.out.println("BALANCE IS " + balance);
    }
}

class SavingsAccount extends Account {
    private double interestrate;
public SavingsAccount(String accountnumber , String accountholder , double balance){
    super(accountnumber, accountholder, balanc
        @Override
        void withdraw(double amount){
            if(amount <= 0){
System.out.println("with draw amount is not possible ");
            }else if (amount >= balance){
System.out.println("it has insufficient balance");
            }else{
                balance -= amount;
System.out.println("withdraw is " + amount );
            }
        }
        @Override
        void calinterest(){
            double interest = balance *interestrate;
            balance += interest;
System.out.println("interest is credited" + interest);
        }
    }

    class CurrentAccount extends Account{
        private double overdraftlimit;
public CurrentAccount(String accountnumber , String accountholder , double balance){
    super(accountnumber, accountholder, balance);
    this.overdraftlimit = overdraftlimit;
    }
    @Override
    void withdraw(double amount){
        if(amount <= 0)
System.out.println("with draw amount is not possible ");
        else if (amount > balance + overdraftlimit){
System.out.println("it has insufficient balance and the limited is done");
        }else{
            balance -= amount;
System.out.println("withdraw is " + amount );
        }
    }
    @Override
    void calinterest(){
        double charge = 50.0;
        balance += charge;
System.out.println("maintainence charge is credited" + charge);
    }
}

    public class LabProgram1 {
        public static void main(String[] args) {
            CurrentAccount ca = new CurrentAccount("UB 24311", "kumar", 4500);
            SavingsAccount sa = new SavingsAccount("sb 24533", "heamnth", 5000);
            System.out.println(" initial account details");
            ca.show();
            System.out.println("");
            sa.show();

            System.out.println("transaction details");
            ca.deposit(2000);
            ca.withdraw(2500);
            ca.calinterest();
            sa.withdraw(3000);
            sa.calinterest();
            sa.transfer(ca, 1000);
            System.out.println("final account details");
            ca.show();
            System.out.println("");
            sa.show();
        }
    }
}

```

OUTPUT OF CODE:

initial account details

ACCOUNT NUMBER IS UB 24311

ACCOUNT HOLDER IS kumar

BALANCE IS 4500.0

ACCOUNT NUMBER IS sb 24533

ACCOUNT HOLDER IS heamnth

BALANCE IS 5000.0

transaction details

the deposited amount is 2000.0

withdraw is 2500.0

maintenance charge is credited 50.0

withdraw is 3000.0

interest is credited 0.0

transfer of 1000.0 from sb 24533 to UB 24311---

withdraw is 1000.0

the deposited amount is 1000.0

final account details

ACCOUNT NUMBER IS UB 24311

ACCOUNT HOLDER IS kumar

BALANCE IS 5050.0

ACCOUNT NUMBER IS sb 24533

ACCOUNT HOLDER IS heamnth

BALANCE IS 1000.0